AI Hackathon Portal

System Design and Build Specification

Event: AI Hackathon, Engineering Faculty, Comilla University
Version 1.0 - 5 November 2026
Audience: Claude (the builder) and the project owner

Purpose. This document fixes every product, design, data and architecture decision for the portal. Build the complete system exactly as described and deliver it as one ZIP file.

1. Builder instructions

- Deliver one ZIP containing the full source code plus README.md, LOCAL_SETUP.md, HOSTING.md, ADMIN_MANUAL.md and ASSUMPTIONS.md.
- The owner is a final-year student who will run this on a laptop first. Every setup step must be copy-paste exact and tested.
- Where this spec is silent, choose the simplest secure option and record it in ASSUMPTIONS.md. Do not stop to ask.
- Everything that can change (dates, fee, team size, scoring scale, number of finalists, weights, contact details) lives in the **settings** table and is editable in the admin panel. The .env file holds only secrets and infrastructure values.
- Automatic email is **out of scope for v1**. Admins send emails manually using generated copy-paste text. Keep an inactive mailer module (provider-agnostic interface) so it can be enabled later.
- Use TypeScript, input validation with zod, SQL migrations, a seed script, and automated tests for the critical flows listed in section 17.
- Follow the design system in section 7 exactly. The site must be fully responsive and attractive. Quality of visual design matters to the owner.

2. Overview and scope

A national-level AI hackathon with 500 to 1000 expected teams from all over Bangladesh. Registration fee is 1000 BDT per team (variable), paid manually by bKash, Nagad or Rocket. The event has two phases: an **online round** (scenario, submission, judging) and an **offline round** (on-site for finalists). The site also advertises side events (seminar, eFootball, exhibition) and sponsors.

In scope (v1)	Out of scope (list as future work)
Public site, team registration with ID photos, manual payment verification, admin-issued team logins, scenario files, submissions, GitHub snapshot and download, judge panel, results release, finalist selection, phase 2 unlock, QR check-in, on-site scoring, final results, admin manual.	Automatic emails, payment gateway, certificates, public live leaderboard, Bangla language toggle, mobile apps, plagiarism detection.

3. Roles and permissions

Role	Access
Visitor	Public pages, registration form, public registered-teams list.
Team (leader login)	Own dashboard. Scenario files during the online window. Submit or edit submission before the deadline. Results after release. Phase 2 page only if the team is a finalist and phase 2 is open.
Judge	Judge panel: sees every team, writes a score and review per team, sees own progress. After phase 2 opens, scores finalists on site. Cannot see payments or ID photos.
Moderator	Verify payments, approve, reject or request payment correction, issue credentials, mark credentials sent, check-in teams on the day.
Super admin	Everything a moderator can do plus settings and dates, staff and judge accounts, scenario documents, deadline overrides, snapshots, lock judging, release results, choose finalists, final results, purge data, audit log.

Staff (super admin, moderator, judge) log in at a hidden URL, **/staff/login**, which is not linked from any public page. Teams log in at **/login** (linked as "Team login" in the header). All permission checks are enforced on the server.

4. Event lifecycle

```
Registration open -> team registers (status: pending)
  -> moderator verifies payment manually:
       approved           : system creates team login, admin sends credentials by email
       payment_correction  : one-time link lets leader fix only the transaction ID, back to pending
```

```
       rejected            : reason stored, admin sends rejection text
Online round (admin sets start and end): approved teams log in, read scenario files, submit
Submission deadline -> snapshot job records commit SHA and downloads repo ZIP for every team
Judging: all judges review all teams; scores 0..max_score plus review; admin locks judging
Admin releases results -> teams see their marks and reviews
Admin marks finalists (is_finalist) -> admin emails them -> phase2_open unlocks their profile
Offline round: QR check-in, on-site scoring by judges, final score and rank computed, admin publishes
```

Every phase is controlled by dates and flags in the settings table. Teams see a countdown for the active phase. All times are stored in UTC and shown in Asia/Dhaka.

5. Functional requirements by module

5.1 Public site

- Landing page sections in order: Hero, About, Schedule, Events, FAQ and Contact, Sponsors, Footer.
- Hero shows event name, date and venue pills, tagline, countdown, two buttons (Register your team, View registered teams) that navigate to separate routes, and a "Powered by" sponsor strip.
- Events section is a bento-style grid (featured AI Hackathon tile plus seminar, eFootball, exhibition tiles) with icons, date chips and status badges. These events are advertising only.
- Sponsors section sits directly before the footer. Logos are managed by replacing files in /public/sponsors or via a simple admin list.
- Header: logo, About, Schedule, Events, Teams, Sponsors, FAQ, Contact, Team login, and a Register now button. Collapses to a menu on mobile. No admin link anywhere.

5.2 Registration page (/register)

- Heading "Register your team", then the payment instruction box (fee and numbers come from settings), then the form.
- Team fields: team name (unique, case-insensitive), university, optional description.
- Member cards: member 1 is the leader and is created automatically and cannot be removed. "Add member" expands a new card with an animation. Same fields for every member: full name, student ID, email, phone, department, year, ID card photo.
- Team size is min_members to max_members from settings (default 3 and 4). The button disables at the maximum and submit is blocked below the minimum. The API and database layer enforce it too.
- ID photo: JPG, PNG or WebP up to 2 MB, preview shown, uploaded straight to private storage with a presigned URL, type and size re-checked on the server.
- Payment: method (bKash, Nagad, Rocket), sender number, mandatory transaction ID (unique per method).
- On success show "Registration received, an admin will verify your payment and contact you". Duplicate protection and clear error messages for duplicate team name, member email or transaction ID.
- Form closes automatically outside registration_open_at and registration_close_at.

5.3 Registered teams page (/teams)

- Searchable, paginated table: team name, university, status badge. Shows pending and approved teams (setting show_pending_teams_public). Never shows emails, phones, IDs or transaction IDs.
- Status updates automatically when a moderator approves.

5.4 Admin and moderator dashboard (/admin)

- Registration queue with filters (status, search by team, leader, transaction ID), counts per status, and pagination.
- Registration detail: members, signed temporary links to ID photos, payment details, history. Actions: Approve, Reject with reason, Request payment correction.
- Approve creates the team login (login email is the leader email, random temporary password, must change at first login). The credentials are shown once with a ready-made email text and Copy button. Regenerate password is available. A "credentials sent" checkbox tracks manual sending.

- Request correction generates a one-time link (valid for a configurable number of days) that the admin copies into an email. The link allows editing only the transaction ID and sender number.
- Settings page for all values in section 9, including every date. Staff page to create judges and moderators with generated passwords. Documents page to upload scenario files. Deadline overrides per team.
- Submissions, snapshots, judging progress, results, finalists, check-in, final results and audit log pages (see sections 11 and 12). CSV export for teams and scores.

5.5 Team portal (/team)

- Dashboard: team status, active phase and countdown, helpline details, and what to do next.
- Scenario documents (guideline, rulebook, problem statements) visible and downloadable only to approved teams between online_start_at and the end of the visibility window.
- Submission form: abstract (required, max 350 characters with live counter), public GitHub repository URL (required), demo video URL (required). Editable until the deadline or a per-team override. All other documents (report, slides, extras) must be in the repo.
- After results are released: team sees its marks and each review. Judges are shown anonymously as Judge A, Judge B (setting).
- Finalists only, when phase2_open is true: phase 2 page with venue, date, instructions and a QR code for check-in.

5.6 Judge panel (/judge)

- Every judge sees every team. List with search, filters (not yet reviewed, reviewed, draft), sort, and a progress bar of own reviews.
- Team view: abstract, repo link, demo video link, snapshot ZIP download link, commit SHA, and the review form: score (0 to max_score, decimals allowed) and a written review. Save draft or Finalize. Editable until judging is locked.
- The team name is visible to judges. Judges never see payment details or ID photos.
- Phase 2: finalist list with on-site score and review form, available only when phase2_open is true.

5.7 Results, finalists and offline round

- Online score of a team is the average of all finalized judge scores. Admin sees ranking, per-judge scores, and who has not finished.
- Admin locks judging, then releases results (teams see marks). "Suggest top N" proposes finalists by rank. Admin edits the list and confirms. The is_finalist attribute drives phase 2 access.
- Admin emails finalists manually (template provided) and turns phase2_open on to unlock their pages.
- Check-in: moderator searches by team name or scans the QR code and records arrival and members present.
- On-site scoring by judges for finalists. Final score = online_weight x online average + onsite_weight x on-site average (defaults 50 and 50, editable). Ties are flagged for the admin to resolve. Admin publishes final results with rank and prize label.

6. Pages and routes

Route	Who	Description
/	Public	Landing page
/register	Public	Team registration form
/teams	Public	Registered teams list
/payment-correction/[token]	Link holder	Fix transaction ID
/login	Teams	Team login
/team	Team	Dashboard, documents, submission, results, phase 2
/staff/login	Staff	Hidden login for admin, moderator, judge

Route	Who	Description
/admin/*	Super admin, moderator	registrations, settings, staff, documents, submissions, snapshots, scores, finalists, checkin, results, audit
/judge/*	Judge	teams list, team review, progress, on-site scoring
/404, /403	All	Branded error pages

7. Design system

Token	Value
Deep background	#0a0618
Violet (brand)	#6d3df5
Cyan (AI accent)	#22d3ee
Amber (call to action)	#ffb020
Paper (light sections)	#f6f4ff
Ink (text)	#1b1236
Line	#e0dbf5
Headings font	Sora (700, 800)
Body font	DM Sans (400, 500, 700)
Radius	10px buttons, 16px cards

- **Hero:** full-height gradient from #0a0618 through #2b1480 and #5a2fd0 to a warm amber edge. A canvas neural-network animation (drifting cyan nodes joined by faint violet lines) sits behind the content. Date and venue pills, large Sora headline, gradient tagline text (cyan to amber), countdown cards with glass effect, two buttons (amber primary, white secondary), sponsor strip, and an SVG wave into the light section.
- **Sections:** alternating paper and deep backgrounds. Cards lift and glow on hover. The Events bento grid uses dark glass tiles with icons.
- **Motion:** scroll reveal (fade and rise, staggered), animated member-card expansion in the registration form, animated status badges, smooth anchor scrolling. Respect prefers-reduced-motion by disabling all non-essential animation.
- **Responsive:** mobile first. Hamburger menu under 900 px. Tables scroll horizontally inside their own container. Touch targets at least 44 px. Test at 360, 768 and 1280 px.
- **Accessibility:** visible focus rings, labels on all inputs, sufficient contrast, semantic landmarks.
- **Dashboards** (admin, judge, team) use the same tokens with a clean light layout, data tables with sticky headers and status badges: pending amber, approved green, rejected red.

8. Architecture and tech stack

Layer	Choice
Frontend	Next.js (React, TypeScript, App Router), Tailwind CSS, Framer Motion for animation
API	Node.js with Express and TypeScript, zod validation, helmet, express-rate-limit
Database	PostgreSQL (managed service in production), migrations with node-pg-migrate or Prisma
File storage	Cloudflare R2 (S3 compatible), private bucket, presigned URLs
Auth	bcrypt password hashes, JWT in httpOnly secure cookies, separate team and staff sessions
Jobs	DB-backed job table with a worker process for repo snapshots (node-cron to schedule)
Email	None in v1. Mailer interface present but disabled.

```
ai-hackathon-portal/
  web/            Next.js app (public site, team portal, admin, judge)
  api/            Express API (routes, services, jobs, middleware)
  db/             migrations/ and seed/ (super admin, sample data)
```

```
  scripts/        export-snapshots, backup, load-test
  docs/           README.md LOCAL_SETUP.md HOSTING.md ADMIN_MANUAL.md ASSUMPTIONS.md
  docker-compose.yml   local Postgres
  .env.example
```

Scale. Expected load is 500 to 1000 teams, with spikes at the registration and submission deadlines. Cache public pages (Next.js revalidate 30 to 60 seconds), paginate every list, index search columns, use a connection pool, upload files directly to R2, and keep the registration write in one short transaction. No microservices.

9. Configuration

9.1 Settings table (editable in admin)

Key	Default	Meaning
event_name, event_venue, event_start_at	AI Hackathon, Engineering Faculty Comilla University, empty	Shown in hero and footer
registration_open_at, registration_close_at	empty	Registration window
registration_fee_bdt	1000	Fee shown on form
bkash_number, nagad_number, rocket_number	empty	Payment instructions
min_members, max_members	3, 4	Team size limits
online_start_at, online_end_at	empty	Scenario visibility and submission window
max_score, score_decimals	100, true	Judge scoring scale
judging_locked, results_released	false, false	Judging and release switches
finalist_count_n	20	Suggested finalists
phase2_open, phase2_date, phase2_venue	false, empty, empty	Offline round unlock
online_weight, onsite_weight	50, 50	Final score weights
show_pending_teams_public	true	Public list shows pending
anonymous_judges_to_teams	true	Judge A, B naming
correction_link_days	3	One-time link validity
id_photo_retention_days	30	Purge reminder after event
support_email, support_phone	empty	Helpline shown everywhere

9.2 Environment variables (.env)

```
NODE_ENV, APP_URL, API_URL, PORT
DATABASE_URL
JWT_SECRET, COOKIE_DOMAIN
R2_ACCOUNT_ID, R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY, R2_BUCKET
GITHUB_TOKEN              (read-only, public repos, used by the snapshot job)
SNAPSHOT_MAX_MB=200, SNAPSHOT_CONCURRENCY=5
INITIAL_SUPER_ADMIN_EMAIL, INITIAL_SUPER_ADMIN_PASSWORD   (used only by the seed script)
```

10. Database schema (PostgreSQL)

```
CREATE TABLE staff (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL, email VARCHAR(120) UNIQUE NOT NULL, password_hash TEXT NOT NULL,
  must_change_password BOOLEAN NOT NULL DEFAULT true,
  role VARCHAR(12) NOT NULL CHECK (role IN ('super_admin','moderator','judge')),
  active BOOLEAN NOT NULL DEFAULT true, failed_logins INT NOT NULL DEFAULT 0, locked_until TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE teams (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_name VARCHAR(60) NOT NULL, university VARCHAR(120) NOT NULL, description TEXT,
  status VARCHAR(20) NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending','payment_correction','approved','rejected')),
  admin_note TEXT, reviewed_by UUID REFERENCES staff(id), reviewed_at TIMESTAMPTZ,
  is_finalist BOOLEAN NOT NULL DEFAULT false, finalist_set_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now());
CREATE UNIQUE INDEX uq_team_name ON teams (lower(team_name));
```

```
CREATE INDEX idx_team_status ON teams (status);

CREATE TABLE team_accounts (
  team_id UUID PRIMARY KEY REFERENCES teams(id) ON DELETE CASCADE,
  login_email VARCHAR(120) UNIQUE NOT NULL, password_hash TEXT NOT NULL,
  must_change_password BOOLEAN NOT NULL DEFAULT true,
  credentials_sent BOOLEAN NOT NULL DEFAULT false, credentials_sent_at TIMESTAMPTZ,
  failed_logins INT NOT NULL DEFAULT 0, locked_until TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE members (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id UUID NOT NULL REFERENCES teams(id) ON DELETE CASCADE,
  is_leader BOOLEAN NOT NULL DEFAULT false,
  full_name VARCHAR(100) NOT NULL, student_id VARCHAR(40) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE, phone VARCHAR(20) NOT NULL,
  department VARCHAR(80), year VARCHAR(20),
  id_card_key TEXT NOT NULL,                -- object key in private R2 bucket
  UNIQUE (team_id, student_id));
CREATE UNIQUE INDEX uq_one_leader ON members (team_id) WHERE is_leader;

CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id UUID NOT NULL UNIQUE REFERENCES teams(id) ON DELETE CASCADE,
  method VARCHAR(10) NOT NULL CHECK (method IN ('bkash','nagad','rocket')),
  sender_no VARCHAR(20) NOT NULL, trx_id VARCHAR(40) NOT NULL, amount NUMERIC(10,2),
  verified_by UUID REFERENCES staff(id), verified_at TIMESTAMPTZ,
  UNIQUE (method, trx_id));

CREATE TABLE correction_tokens (
  token_hash TEXT PRIMARY KEY, team_id UUID NOT NULL REFERENCES teams(id) ON DELETE CASCADE,
  expires_at TIMESTAMPTZ NOT NULL, used_at TIMESTAMPTZ);

CREATE TABLE settings (
  key TEXT PRIMARY KEY, value JSONB NOT NULL,
  updated_by UUID REFERENCES staff(id), updated_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE documents (                    -- scenario files uploaded by super admin
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(), title VARCHAR(120) NOT NULL,
  kind VARCHAR(20) NOT NULL CHECK (kind IN ('guideline','rulebook','problem_statement','other')),
  file_key TEXT NOT NULL, file_name TEXT NOT NULL, size_bytes BIGINT,
  visible BOOLEAN NOT NULL DEFAULT true, sort_order INT NOT NULL DEFAULT 0,
  uploaded_by UUID REFERENCES staff(id), created_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE deadline_overrides (
  team_id UUID PRIMARY KEY REFERENCES teams(id) ON DELETE CASCADE,
  online_end_at TIMESTAMPTZ NOT NULL, reason TEXT, set_by UUID REFERENCES staff(id));

CREATE TABLE submissions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id UUID NOT NULL UNIQUE REFERENCES teams(id) ON DELETE CASCADE,
  abstract VARCHAR(350) NOT NULL CHECK (char_length(abstract) <= 350),
  repo_url TEXT NOT NULL, video_url TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(), updated_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE repo_snapshots (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  submission_id UUID NOT NULL UNIQUE REFERENCES submissions(id) ON DELETE CASCADE,
  commit_sha CHAR(40), branch TEXT, captured_at TIMESTAMPTZ,
  zip_key TEXT, zip_size_bytes BIGINT,
  status VARCHAR(20) NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending','ok','failed','repo_not_public','too_large')),
  error TEXT, attempts INT NOT NULL DEFAULT 0);

CREATE TABLE reviews (                      -- online round, one per judge per submission
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  submission_id UUID NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
  judge_id UUID NOT NULL REFERENCES staff(id),
  score NUMERIC(6,2) NOT NULL CHECK (score >= 0),   -- upper bound = max_score, checked in API
  comment TEXT NOT NULL, is_final BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(), updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (submission_id, judge_id));

CREATE TABLE onsite_scores (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id UUID NOT NULL REFERENCES teams(id) ON DELETE CASCADE,
  judge_id UUID NOT NULL REFERENCES staff(id),
  score NUMERIC(6,2) NOT NULL CHECK (score >= 0), comment TEXT,
  is_final BOOLEAN NOT NULL DEFAULT false, updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (team_id, judge_id));

CREATE TABLE checkins (
  team_id UUID PRIMARY KEY REFERENCES teams(id) ON DELETE CASCADE,
```

```
  checked_in_at TIMESTAMPTZ NOT NULL DEFAULT now(), checked_in_by UUID REFERENCES staff(id),
  members_present INT);

CREATE TABLE final_results (
  team_id UUID PRIMARY KEY REFERENCES teams(id) ON DELETE CASCADE,
  online_avg NUMERIC(6,2), onsite_avg NUMERIC(6,2), final_score NUMERIC(6,2),
  rank INT, prize_label VARCHAR(60), published BOOLEAN NOT NULL DEFAULT false);

CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY, actor_type VARCHAR(10), actor_id UUID, action TEXT NOT NULL,
  target_type TEXT, target_id TEXT, detail JSONB, created_at TIMESTAMPTZ NOT NULL DEFAULT now());
```

Notes: the team, members and payment are inserted in one transaction. Unique constraints are the final defence against duplicate transaction IDs, team names and member emails. Team size and score limits come from settings and are enforced in the API (not hardcoded in the schema). Team accounts never share tables with staff accounts.

11. API endpoints

Method and path	Access	Purpose
GET /api/public/config	Public	Event info, dates, fee, team size, flags
GET /api/public/teams	Public	Paginated search: name, university, status only
POST /api/uploads/id-card-url	Public, rate limited	Presigned PUT for an ID photo
POST /api/register	Public, rate limited	Create team, members, payment (transaction)
POST /api/payment-correction/:token	Token	Update transaction ID, status back to pending
POST /api/auth/team/login, /api/auth/staff/login	Public	Login with lockout after repeated failures
POST /api/auth/logout, /change-password; GET /api/auth/me	Logged in	Session handling
GET /api/team/dashboard	Team	Status, phase, countdown
GET /api/team/documents, /documents/:id/download	Team, in window	Scenario files via signed URL
GET, PUT /api/team/submission	Team, before deadline	Read and save abstract, repo URL, video URL
GET /api/team/results	Team, after release	Marks and reviews
GET /api/team/phase2, /api/team/qr	Finalist, phase2_open	Phase 2 info and QR
GET /api/admin/registrations, /registrations/:id	Moderator, admin	Queue and detail with signed ID photo URLs
POST /api/admin/registrations/:id/approve | reject | request-correction	Moderator, admin	Decision actions (approve returns credentials once)
POST .../regenerate-credentials, .../mark-credentials-sent	Moderator, admin	Credential handling
GET, PUT /api/admin/settings	Super admin	All settings and dates
GET, POST, PATCH /api/admin/staff	Super admin	Create judges and moderators, reset password, deactivate
GET, POST, PATCH, DELETE /api/admin/documents	Super admin	Scenario files (upload via presigned URL)
PUT /api/admin/deadline-overrides/:teamId	Super admin	Extend deadline for one team
GET /api/admin/submissions; GET /api/admin/snapshots	Super admin	Submissions and snapshot status
POST /api/admin/snapshots/run, /snapshots/:id/retry	Super admin	Run or retry snapshot job
GET /api/admin/snapshots/export.csv, /export-script	Super admin	CSV of signed ZIP links and a bulk download script
GET /api/admin/scores	Super admin	Per-team averages, per-judge scores, progress
POST /api/admin/judging/lock, /results/release	Super admin	Lock judging, release results
POST /api/admin/finalists/suggest, PUT /api/admin/finalists	Super admin	Suggest top N, save finalist flags

Method and path	Access	Purpose
GET /api/admin/checkin/lookup; POST /api/admin/checkin/:teamId	Moderator, admin	Check-in by name or QR
POST /api/admin/final-results/compute, /publish	Super admin	Final score, rank, publish
GET /api/admin/export/teams.csv; GET /api/admin/audit	Super admin	Exports and audit log
POST /api/admin/purge-id-photos	Super admin	Delete ID photos from storage
GET /api/judge/teams, /teams/:id, /progress	Judge	All teams, team detail, own progress
PUT /api/judge/reviews/:submissionId	Judge, not locked	Save draft or finalize score and review
GET /api/judge/finalists; PUT /api/judge/onsite/:teamId	Judge, phase2_open	On-site scoring

12. GitHub snapshot and download subsystem

Goal: freeze every team's work at the deadline so later pushes cannot change what judges see, and give the admin and judges a downloadable ZIP per team without clicking each repository.

- **Trigger:** a scheduled job runs at online_end_at (plus each team's override date) and the admin can run or retry it manually.
- **Per submission:** parse owner and repo from repo_url. Call GET /repos/{owner}/{repo} with GITHUB_TOKEN. If the repo is missing or private, set status repo_not_public.
- Find the last commit at or before the deadline: GET /repos/{owner}/{repo}/commits?until=DEADLINE_ISO&per;_page=1. Store commit_sha, branch and captured_at.
- Download GET /repos/{owner}/{repo}/zipball/{sha}, stream it to R2 at snapshots/TEAM_ID/SHA.zip. Enforce SNAPSHOT_MAX_MB, otherwise status too_large.
- **Reliability:** concurrency from SNAPSHOT_CONCURRENCY, retries with backoff, respect GitHub rate limits (5000 requests per hour with a token), idempotent re-runs, every failure stored in the error column.
- **Admin UI:** table of all teams with snapshot status, commit link, size, retry button, and failed-only filter. Export CSV with signed ZIP links and a generated script that downloads all ZIPs in one command.
- **Judges** see the commit SHA, the repo link and a signed ZIP download link on each team page. They may review the live repo or the ZIP.
- Known limits to document: GitHub ZIPs exclude Git LFS files and submodules; if a repo is deleted before the job runs, the snapshot fails and the admin is alerted in the dashboard.

13. Security and privacy

- Passwords hashed with bcrypt (cost 12). Temporary passwords are random, shown once, and must be changed at first login. Lockout after 5 failed logins for 15 minutes.
- httpOnly, secure, SameSite cookies. CSRF protection on state-changing routes. helmet headers. CORS restricted to APP_URL.
- Role checks on every API route on the server. Staff login is not linked publicly. Hidden pages are not treated as security.
- Rate limits on registration, login and upload-URL endpoints. Optional CAPTCHA hook on registration.
- Private R2 bucket only. Signed URLs expire in 10 minutes. Random object keys. Validate file type by content (magic bytes) and size on the server.
- Public APIs return minimum data. Emails, phones, IDs, transaction IDs and photos never appear in public responses.
- ID photos are personal data. Only moderators and the super admin can view them. Provide the purge action and a reminder after id_photo_retention_days.
- Audit log records every approval, rejection, setting change, credential issue, score lock, release and purge.

- Database backups daily with a tested restore procedure. Secrets only in environment variables, never in the repository.

14. Hosting guide (to be expanded into HOSTING.md)

Option A: managed services (recommended)

Part	Service	Notes
Frontend	Vercel (or Netlify)	Connect the GitHub repo, set env vars, custom domain with HTTPS
API and worker	Render or Railway	One web service and one background worker, health check route /health
Database	Neon or Supabase Postgres (database only)	Enable daily backups, use the pooled connection string
Files	Cloudflare R2	Private bucket, CORS allowing PUT from APP_URL, access keys scoped to the bucket

Option B: one VPS or university server

- Ubuntu LTS, Node.js LTS, PostgreSQL, Nginx reverse proxy, PM2 or systemd for web, api and worker, Certbot for HTTPS, UFW firewall.
- Nightly pg_dump to a second location plus weekly restore test.
- Use this when the faculty provides a server and a domain.

Go-live checklist

- Domain and HTTPS working, all .env values set, migrations run, seed script creates the super admin, default password changed.
- R2 CORS configured and a test upload works. GitHub token tested against a sample repo.
- Settings filled: dates, fee, payment numbers, contact details, sponsors, scenario files.
- Load test the registration and public pages with 200 concurrent users before opening.
- Backups confirmed. Monitoring: uptime check on /health and error logging.

15. Admin manual (summary for ADMIN_MANUAL.md)

Step	What the admin does
1. Prepare	Super admin logs in, changes the password, fills settings and dates, uploads scenario documents (hidden until the window), creates judges and moderators and shares their credentials.
2. Registration	Open the queue daily. Compare each transaction ID with the bKash, Nagad or Rocket statement. Approve, reject with a reason, or request correction.
3. Credentials	After approval copy the generated email text with login details, send it from the official mailbox, tick "credentials sent".
4. Online round	Scenario unlocks at online_start_at. Monitor submissions. Use deadline overrides only with a recorded reason.
5. Snapshots	After the deadline check the snapshot table, retry failures, then export ZIP links for judges.
6. Judging	Track judge progress. Lock judging when complete. Review averages, then release results.
7. Finalists	Use Suggest top N, adjust, save. Email finalists with the template. Turn on phase2_open.
8. Offline day	Moderators check teams in by name or QR. Judges score on site. Compute and publish final results.
9. After event	Export data, purge ID photos, switch off sensitive access, archive a database backup.

Email templates (generated by the system, copied by the admin)

```
Subject: AI Hackathon - registration confirmed
Dear LEADER_NAME, your team "TEAM_NAME" is confirmed. Log in at APP_URL/login
Email: LOGIN_EMAIL   Temporary password: TEMP_PASSWORD (you must change it at first login).
```

```
Helpline: SUPPORT_EMAIL, SUPPORT_PHONE. Organizing Committee

Subject: AI Hackathon - payment correction needed
Dear LEADER_NAME, we could not verify the transaction ID for "TEAM_NAME". Reason: REASON.
Update it here (valid until DATE): APP_URL/payment-correction/TOKEN

Subject: AI Hackathon - registration not approved       (reason included)
Subject: AI Hackathon - you are a finalist              (venue, date, how to find the phase 2 page)
```

16. Testing checklist

- Registration: min and max member limits, duplicate team name, duplicate transaction ID, duplicate member email, wrong file type, file over 2 MB, closed registration window.
- Approval flow: approve creates account once, credentials shown once, regenerate works, forced password change works, rejected and correction paths work, correction link works once only.
- Access control: a team cannot read another team's data, judges cannot see payments, moderators cannot change settings, hidden staff routes reject unauthenticated users.
- Time rules: documents hidden before online_start_at, submissions blocked after deadline, override extends only the chosen team.
- Submission: abstract limit of 350, invalid GitHub URL rejected, edit before deadline works.
- Snapshots: public repo ok, private repo flagged, deleted repo flagged, oversize flagged, retry works, ZIP link signed and expiring.
- Judging: score outside 0 to max_score rejected, one review per judge per team, draft versus final, lock blocks edits, averages correct.
- Results: teams see marks only after release. Finalist flag controls phase 2 access. QR check-in works. Final score and ranking correct, ties flagged.
- Responsive and accessibility checks at 360, 768 and 1280 px. Reduced-motion mode. Load test of public pages and registration.

17. Build order for the builder

Milestone	Deliverable and acceptance
M0 Scaffold	Monorepo, docker-compose Postgres, migrations, seed, settings loader, lint and test setup.
M1 Public site	Landing page with hero, sections, theme and animation, header and footer, teams page. Responsive.
M2 Registration	Dynamic member form, ID upload to R2, transaction, validations, success state.
M3 Auth and admin core	Team and staff login, roles, registration queue, approve, reject, correction link, credentials text.
M4 Settings and documents	Settings page with dates, staff management, scenario document upload and time-gated team access.
M5 Team portal and submission	Dashboard, documents, submission form, countdowns, overrides.
M6 Snapshots	GitHub snapshot job, admin table, export CSV and script, retry handling.
M7 Judge panel	Team list, review form, progress, lock, averages, release, team results view.
M8 Finalists and phase 2	Suggest top N, finalist flag, phase 2 page, QR, check-in, on-site scoring, final results.
M9 Hardening and docs	Security pass, rate limits, tests, load test, audit log, README, LOCAL_SETUP, HOSTING, ADMIN_MANUAL, ASSUMPTIONS.

18. Future work (not in v1)

- Automatic transactional email with a provider such as Brevo or Resend, using the existing mailer interface.
- Bulk payment verification from an uploaded bKash or Nagad statement CSV.
- Certificates, public live leaderboard, Bangla language toggle, plagiarism checks, sponsor self-service.